

Phase 2 잔여항목 설계서

1. 논리/물리 모델 구분 관리

1.1 목적

하나의 데이터 모델에서 논리 모델(엔티티/속성)과 물리 모델(테이블/컬럼)을 구분하여 관리하고, 논리→물리 매핑을 제공한다.

1.2 현재 상태

- TB_DATA_MODEL_ATTR에 테이블/컬럼 정보만 저장 (물리 모델만)
- 논리 모델(엔티티명, 속성명) 개념이 없음
- ERwin 임포트에서 논리명(name)과 물리명(physical_name)을 모두 가져오지만, 한글명(attrNmKr)에 논리명을 넣는 방식

1.3 설계

테이블 변경

TB_DATA_MODEL_OBJ, TB_DATA_MODEL_ATTR에 논리명 컬럼이 이미 있는지 확인 필요.

```
-- TB_DATA_MODEL_OBJ에 이미 OBJ_NM_KR이 있음 → 이것을 논리 엔티티명으로 활용
-- TB_DATA_MODEL_ATTR에 이미 ATTR_NM_KR이 있음 → 이것을 논리 속성명으로 활용

-- 추가 필요 컬럼
ALTER TABLE TB_DATA_MODEL ADD COLUMN IF NOT EXISTS MODEL_TYPE VARCHAR(10) DEFAULT 'PHYSICAL';
-- MODEL_TYPE: PHYSICAL(물리만), LOGICAL(논리만), BOTH(논리+물리)
```

화면 변경

데이터 모델 현황에서 논리/물리 전환 탭 또는 토글:

데이터 모델 현황 - 회사오라클테스트

[물리 모델] [논리 모델] ← 탭 전환

물리 모델 선택 시:

테이블명(물리)	테이블명(논리)	컬럼 수
TB_USER	사용자	15

논리 모델 선택 시:

엔터티명(논리)	테이블명(물리)	속성 수
사용자	TB_USER	15

핵심: 이미 있는 데이터 활용

사실 OBJ_NM_KR(테이블 한글명)이 논리 엔터티명이고, ATTR_NM_KR(컬럼 한글명)이 논리 속성명이다.

별도 테이블 생성 없이, 기존 데이터를 논리/물리 관점으로 보여주는 뷰 전환만 추가하면 된다.

구현 범위

항목	내용	난이도
TB_DATA_MODEL에 MODEL_TYPE 컬럼 추가	PHYSICAL/LOGICAL/BOTH	낮음
데이터 모델 현황에 논리/물리 탭 전환	컬럼 표시 순서 변경	중
하단 테이블/컬럼 상세에서 논리명 강조	논리 탭일 때 논리명 먼저 표시	낮음
ERwin 임포트에서 MODEL_TYPE 자동 설정	논리+물리 모두 있으면 BOTH	낮음

2. 통합 검색

2.1 목적

헤더 영역에 검색창을 추가하여, 단어/용어/도메인/코드/테이블/컬럼을 한 번에 검색한다.

2.2 현재 상태

- 각 메뉴(단어, 용어, 도메인 등)에서 개별 검색만 가능
- 전체를 관통하는 검색 기능 없음

2.3 설계

검색 UI

헤더 우측에 검색 아이콘 또는 검색 입력창:

[로고]



[통합 검색...]



클릭 시 검색 결과 화면이 열림 (별도 탭 또는 모달):

통합 검색 결과: "사용자"

단어 (3건)

사용자	USR	USER
사용자ID	USRID	USER ID
사용자명	USRNM	USER NAME

용어 (5건)

사용자이름	USR_NM	V100
사용자ID	USR_ID	V20
...		

도메인 (0건)

테이블/컬럼 (12건)

TB_USER	USER_ID	사용자ID	오라클
TB_USER	USER_NM	사용자명	오라클
...			

API

GET /api/std/search?keyword=사용자

응답:

```
{
  "words": [...],      // 단어명 또는 영문약어에 키워드 포함
  "terms": [...],      // 용어명 또는 영문약어에 키워드 포함
  "domains": [...],    // 도메인명에 키워드 포함
  "codes": [...],      // 코드명에 키워드 포함
  "columns": [...],    // 테이블명 또는 컬럼명에 키워드 포함
}
```

MyBatis 쿼리

```
-- 단어 검색
SELECT WORD_ID, WORD_NM, WORD_ENG_ABRV_NM, WORD_ENG_NM
FROM TB_WORD
WHERE APRV_YN = 'Y'
      AND (WORD_NM ILIKE '%' || #{keyword} || '%'
            OR WORD_ENG_ABRV_NM ILIKE '%' || #{keyword} || '%')
LIMIT 20;

-- 용어 검색
SELECT TERMS_ID, TERMS_NM, TERMS_ENG_ABRV_NM, DOMAIN_NM
FROM TB_TERMS
WHERE APRV_YN = 'Y'
      AND (TERMS_NM ILIKE '%' || #{keyword} || '%'
            OR TERMS_ENG_ABRV_NM ILIKE '%' || #{keyword} || '%')
LIMIT 20;

-- 도메인 검색
SELECT DOMAIN_ID, DOMAIN_NM, DOMAIN_ENG_NM, DATA_TYPE, DATA_LEN
FROM TB_DOMAIN
WHERE DOMAIN_NM ILIKE '%' || #{keyword} || '%'
LIMIT 20;

-- 테이블/컬럼 검색 (최신 수집 기준)
SELECT a.OBJ_NM, a.ATTR_NM, a.ATTR_NM_KR, dm.DM_NM
FROM TB_DATA_MODEL_ATTR a
JOIN TB_DATA_MODEL dm ON a.DM_ID = dm.DM_ID
WHERE (a.OBJ_NM ILIKE '%' || #{keyword} || '%'
       OR a.ATTR_NM ILIKE '%' || #{keyword} || '%'
       OR a.ATTR_NM_KR ILIKE '%' || #{keyword} || '%')
LIMIT 20;
```

구현 범위

항목	내용	난이도
MyBatis 쿼리 (search.xml)	4~5개 UNION 또는 개별 쿼리	낮음
API 1개	/api/std/search?keyword=	낮음
헤더에 검색 입력창	NdHaeder.vue 수정	중
검색 결과 화면	DSGlobalSearch.vue 신규	중
결과 클릭 → 해당 메뉴 이동	단어 클릭 → 단어 화면으로	중

3. 멀티 DBMS 전체 기능 검증

3.1 목적

현재 수집은 9종 DBMS를 지원하지만, 전체 기능(표준화 진단, 구조 진단 등)이 Oracle 외 DBMS에서도 정상 동작하는지 검증한다.

3.2 검증 대상

DBMS	JDBC 드라이버	검증 필요 항목
Oracle	ojdbc8	기준 (검증 완료)
PostgreSQL	postgresql-42.x	수집, 표준화 진단, 구조 진단
MariaDB/MySQL	mariadb-java-client	수집, 표준화 진단, 구조 진단
MSSQL	mssql-jdbc	수집, 표준화 진단, 구조 진단
Tibero	tibero6-jdbc	수집, 표준화 진단, 구조 진단

3.3 검증 방법

3.3.1 자동화 테스트 스크립트

각 DBMS별 Docker 컨테이너를 띄우고: 1. 데이터소스 등록 2. 샘플 테이블/컬럼 생성 3. 수집 실행 4. 표준화 진단 실행 5. 구조 진단 실행 6. 결과 검증

3.3.2 수동 검증 체크리스트

#	테스트 항목	Oracle	PostgreSQL	MariaDB	MSSQL	Tibero
1	데이터소스 연결	✓				
2	스키마 수집 (테이블 목록)	✓				
3	컬럼 수집 (타입/길이/nullable)	✓				
4	표준화 진단 실행	✓				
5	구조 진단 (라이브 비교)	✓				
6	DDL 생성	✓				

3.4 예상 이슈

DBMS	예상 이슈	대응
PostgreSQL	스키마 개념 차이 (schema.table)	이미 지원 중
MariaDB	information_schema 쿼리 차이	GetAttrs 쿼리 확인 필요
MSSQL	데이터 타입 명칭 차이 (nvarchar 등)	타입 동의어 확장
Tibero	Oracle 호환이므로 이슈 적을 것	검증만

3.5 구현 범위

항목	내용	난이도
GetAttrs/GetObjs 쿼리 확인	각 DBMS별 수집 쿼리가 이미 있는지 확인	낮음
타입 동의어 확장 (DiagService)	nvarchar↔varchar, int↔integer 등	낮음
Docker 테스트 환경 구축	각 DBMS 컨테이너	중
검증 체크리스트 실행	수동 테스트	중
이슈 수정	발견된 호환성 문제	상황에 따라

4. 전체 우선순위

순서	기능	난이도	예상 시간	비고
1	통합 검색	중	2시간	사용자 편의성 즉시 향상
2	논리/물리 모델 구분	낮~중	1시간	기존 데이터 활용, 뷰 전환만
3	멀티 DBMS 검증	중	테스트 환경 의존	Docker 필요, 순차 진행